🌍🟦🟩 **LOAD BALANCER BLUE-GREEN DEPLOYMENT**
4-Year DevOps Engineer – REAL-TIME JOB STYLE
Zero Knowledge → Production Level

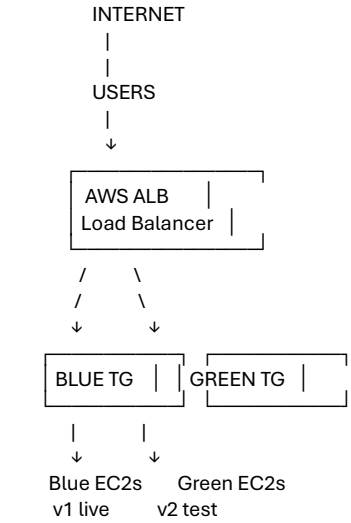
🧠 **FIRST: BLUE-GREEN ASALU ENTI?**
Imagine oka shopping website:
www.myshop.com
Currently customers website use chestunnaru.
Current version:
v1
Now developer team new version:
v2
deploy cheyyali.
Problem enti?
Normal deployment:
Stop v1
↓
Deploy v2
↓
Start v2
Ee time lo:
Users → Website ❌
Downtime.
Blue-Green deployment lo:
BLUE = Current Production
GREEN = New Version
Rendu simultaneously run chestayi.

```
        ┌──→ BLUE v1
Users → Load Balancer
        └──→ GREEN v2
```

Initially:
Users
↓
Load Balancer
↓
BLUE v1
Green ni users ki direct ga ivvam.
First Green test chestam.
Everything okay anipisthe:
Users
↓
Load Balancer
↓
GREEN v2
Problem vaste:
Users
↓
Load Balancer
↓
BLUE v1
So main idea:
New version ni first separate environment lo ready chesi test chesi, traffic ni switch cheyyadam.

🟦 **BLUE &** 🟩 **GREEN ENDuku?**
Simple example.
Suppose restaurant lo:
Blue kitchen
Already running.
Blue = Old kitchen
Customers = already eating
New kitchen build chestunnam:
Green = New kitchen
First Green kitchen lo food test chestam.
Food okay:
Customers → Green
Food problem:
Customers → Blue
Customer ki almost no interruption.
That's exactly Blue-Green deployment.

🏗️ COMPLETE ARCHITECTURE



Here:
ALB = Application Load Balancer
TG = Target Group
Target Group ante simple ga:
Load Balancer traffic pampinche servers list.
Example:
tg-blue

EC2-1
EC2-2
EC2-3
Green:
tg-green

EC2-4
EC2-5
EC2-6

🔥 REAL-TIME JOB SCENARIO

Suppose nenu 4 years experience unna DevOps engineer.
Developer cheptadu:
"v2 login feature ready. Production lo zero downtime tho deploy cheyyali."
Na work:
1. Existing Blue verify
2. Green create
3. v2 deploy
4. Green test
5. Green health verify
6. Load Balancer configure
7. Traffic switch
8. Monitor
9. Problem → rollback Blue
10. Success → Green becomes production
Let's do each step.

1️⃣ MORNING – EXISTING PRODUCTION CHECK

First production already healthy unda check chestha.
AWS Console:
EC2
↓
Instances
Check:
Blue EC2
Status = Running
Health = Passed
Then:
Target Groups
↓
tg-blue
Check:

Target 1 → healthy
Target 2 → healthy
Why?
Suppose Blue itself already unhealthy.
Manam Green deploy chesi traffic switch chesina, problem deployment valla vachinda or existing infrastructure valla vachinda teliyadu.
Experienced engineer first baseline check chestadu.

---

2 BLUE CURRENT PRODUCTION
Assume:
Blue EC2
Private IP: 10.0.1.10
App: v1
Users:
Users
↓
ALB
↓
tg-blue
↓
Blue EC2
Everything healthy.

---

3 GREEN ENVIRONMENT CREATE
Now new EC2 instances create chestam.
Example:
Green EC2
AMI: Amazon Linux
Instance: suitable production size
Security Group: same required application rules
Subnet: appropriate application subnet
Real production lo t2.micro blindly use cheyyakudadhu.
Instance size workload based decide chestam.
Example:
Blue:
2 × m6i.large

Green:
2 × m6i.large
Why same capacity?
Because Blue handle chestunna production traffic ni Green kuda handle cheyyagaladani ensure cheyyali.

---

4 GREEN EC2 SECURITY GROUP
Green server ki required traffic matrame allow cheyyali.
Typical architecture:
Internet
↓
ALB
↓
Green EC2
So ideally Green EC2 port 80/443 access:
Source = ALB Security Group
not:
0.0.0.0/0
for internal application traffic.
SSH:
Port 22
Source = company VPN / bastion / approved IP
Not:
0.0.0.0/0
because that exposes SSH to the entire internet.

---

5 SSH INTO GREEN
Example:
ssh -i key.pem ec2-user@GREEN_PUBLIC_IP
If production architecture uses private subnet, public IP undakapovachu.
Then:
User
↓
Bastion / SSM
↓

Private Green EC2
Experienced environments lo **AWS Systems Manager Session Manager** often preferred over opening SSH publicly.

6️⃣ INSTALL APPLICATION

Suppose Nginx application.
sudo dnf install nginx -y
Start:
sudo systemctl enable --now nginx
Check:
systemctl status nginx
Expected:
Active: active (running)

7️⃣ DEPLOY V2

Example:
cd /usr/share/nginx/html
Test page:
<h1>GREEN VERSION - V2</h1>
Production application lo actual process usually:
Git
↓
CI/CD
↓
Docker image
↓
EC2 / Kubernetes
Manual vi index.html is only a simple demonstration.

8️⃣ GREEN LOCAL HEALTH CHECK

First server itself nundi test:
curl http://localhost
Expected:
GREEN VERSION - V2
Then application health endpoint:
curl http://localhost/health
Expected:
200 OK
If API application:
curl http://localhost:8080/health

9️⃣ WHY /health IMPORTANT?

Load Balancer ki question:
"Ee server healthy ga unda?"
Application answer:
GET /health
If:
HTTP 200
ALB says:
Healthy ✅
If:
HTTP 500
ALB says:
Unhealthy ❌
Then ALB traffic avoid chestundi.

🔟 TEST GREEN WITHOUT USERS

This is very important.
Green ready ayina immediately public traffic ivvakudadhu.
First test:
Green EC2
↓
Application
↓
Database
↓
External APIs
Check:
Frontend
Homepage
Login

Logout
Search
Backend
GET /health
GET /users
POST /login
Database
DB connection
Read
Write
External dependencies
Payment
Email
Third-party APIs

1️⃣1️⃣ DATABASE IS THE BIG BLUE-GREEN PROBLEM

This is where experienced engineer thinking matters.
Suppose v2 needs DB change:
v1 DB schema
New v2 expects:
v2 DB schema
If immediately DB schema change chesthe:
Blue v1 ❌
Green v2 ✅
Blue rollback kuda fail avvachu.
So production deployments lo often:
Expand → Migrate → Contract
First backward-compatible DB changes.
Example:
Add new column
without immediately removing old column.
Then:
Blue v1
Green v2
both work.
After successful migration:
remove old column later
This is one of the important real-world Blue-Green considerations.

1️⃣2️⃣ CREATE BLUE TARGET GROUP

AWS:
EC2
↓
Target Groups
↓
Create target group
Example:
Name:
tg-blue
Protocol:
HTTP
Port:
80
Register:
Blue EC2 instances
Health check:
HTTP
Path:
/health

1️⃣3️⃣ CREATE GREEN TARGET GROUP

Same process:
tg-green
Register:
Green EC2
Green EC2
Health check:
/health
Wait until:
Green Target 1 → Healthy

Green Target 2 → Healthy
Only then traffic switch consider cheyyali.

1️⃣4️⃣ ALB

Architecture:
Users
↓
ALB
ALB listener:
HTTP : 80
or production:
HTTPS : 443
HTTPS is preferred for public applications.

1️⃣5️⃣ CURRENT LISTENER

Initially:
ALB
↓
Listener 80
↓
tg-blue
So:
Users → ALB → tg-blue → Blue
Green:
Green = Healthy
Green = Test
Users still Blue.

🔥 1️⃣6️⃣ FINAL PRE-SWITCH CHECKLIST

Traffic switch before check:
☑ Green instances healthy
☑ /health returning 200
☑ Application starts correctly
☑ Database connection okay
☑ APIs tested
☑ Logs clean
☑ CPU okay
☑ Memory okay
☑ Security group correct
☑ ALB health check healthy
☑ Rollback plan ready
☑ Blue still healthy
This is what differentiates:
"commands know"
from
"production engineer".

🔥 1️⃣7️⃣ TRAFFIC SWITCH

Before:
Users
↓
ALB
↓
tg-blue
↓
Blue v1
Change listener:
tg-blue
to:
tg-green
After:
Users
↓
ALB
↓
tg-green
↓
Green v2
Now v2 is production.

🧠 **IMPORTANT: "ZERO DOWNTIME" MEANS WHAT?**
It doesn't mean:
Absolutely nothing can ever go wrong.
It means:
Traffic is moved to an already-running healthy environment instead of stopping the old environment first.
So Blue remains available while Green is prepared.

1️⃣8️⃣ **IMMEDIATELY AFTER SWITCH**
Don't just say:
Done.
Start monitoring.
Check:
ALB
↓
Target Group
↓
Green
Monitor:
Request count
HTTP 4xx
HTTP 5xx
Target response time
Healthy hosts
Unhealthy hosts

1️⃣9️⃣ **CHECK APPLICATION LOGS**
Nginx:
tail -f /var/log/nginx/access.log
Errors:
tail -f /var/log/nginx/error.log
Application-specific logs:
journalctl -u myapp -f
Docker app అయితే:
docker logs -f <container>
Kubernetes అయితే:
kubectl logs -f <pod>

🚨 2️⃣0️⃣ **REAL-TIME INCIDENT**
Suppose traffic switch ayyaka:
Users → Green
Suddenly:
HTTP 500 ↑
CloudWatch:
5XX = increasing
Users complain:
"Login not working."
Engineer immediately investigates.

2️⃣1️⃣ **CHECK GREEN LOGS**
tail -f /var/log/nginx/error.log
Application log:
Database connection timeout
Then investigate:
Green → DB
Maybe:
Security Group ❌
Environment variable ❌
DB credentials ❌
Wrong DB endpoint ❌

🔥 2️⃣2️⃣ **ROLLBACK**
Don't spend 30 minutes debugging while users are impacted.
First:
Restore service
Switch:
tg-green
↓
tg-blue
Now:
Users

↓
ALB
↓
Blue
↓
v1
Service restored.
Then investigate Green separately.
This is production mindset:
Restore first, investigate second, when user impact is significant.

🟦🟩 2️⃣3️⃣ BLUE IS NOT IMMEDIATELY DELETED

Very important.
After successful deployment:
Green = Production
Blue = Rollback environment
Don't immediately terminate Blue.
Keep it for an agreed stabilization period.
Example:
Green v2 → live
Blue v1 → standby
If v2 problem:
ALB → Blue
Rollback fast.

2️⃣4️⃣ AFTER STABILIZATION

Suppose:
v2 stable for several hours/day
Then:
Blue v1
can be decommissioned according to team process.
But production doesn't normally mean:
delete immediately
Need consider:
rollback window
cost
capacity
compliance
incident risk

👷 REAL 4-YEAR DEVOPS DAILY WORKFLOW

Your actual day might look like:
09:00
↓
Check Jira
↓
Check production dashboards
↓
Check ALB
↓
Check EC2
↓
Check alerts
↓
Review deployment request
↓
Pull code / image
↓
Prepare Green
↓
Deploy v2
↓
Health check
↓
Functional testing
↓
Traffic switch
↓
Monitor
↓
If stable → keep Green

↓
If issue → rollback Blue
↓
Update Jira
↓
Document deployment

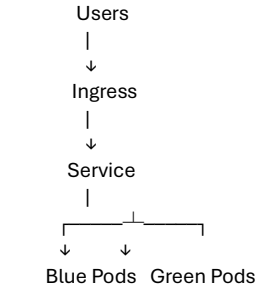
🟦🟩 **DNS BLUE-GREEN**
Now difference.
ALB approach:
Users
↓
ALB
↓
Blue / Green
DNS approach:
Users
↓
DNS
↓
Blue / Green
Example:
myapp.com → Blue
Change:
myapp.com → Green
Problem:
DNS records are cached according to TTL and resolver behavior.
So transition can be gradual.
Some users may still reach Blue.
Some may reach Green.
Therefore DNS switch is **not the same as an instant guaranteed cutover**.

🧠 **DNS BLUE-GREEN EXAMPLE**
Before:
myapp.com
↓
Blue EC2
Green:
green.myapp.com
↓
Green EC2
Test:
green.myapp.com
Everything okay.
Change DNS:
myapp.com
↓
Green
But:
User A → Green
User B → Blue
may happen during DNS propagation/cache periods.
Rollback:
myapp.com → Blue
Again, cache means rollback isn't necessarily instant.
So:
ALB Blue-Green
Traffic switch = fast
Rollback = fast
DNS Blue-Green
Traffic switch = cache dependent
Rollback = cache dependent

☸️ **KUBERNETES BLUE-GREEN**
Kubernetes lo concept same.
Instead of EC2:
Blue Pods
Green Pods
Example:
app-blue
replicas = 3

image = myapp:v1
Green:
app-green
replicas = 3
image = myapp:v2

☸️ **KUBERNETES ARCHITECTURE**



But Service usually points to one version at a time for strict Blue-Green.
Blue:
selector:
app: myapp
version: blue
Green:
selector:
app: myapp
version: green
Service currently:
selector:
app: myapp
version: blue

🔥 **K8S TRAFFIC SWITCH**

Change selector:
blue
to:
green
Now:
Users
↓
Service
↓
Green Pods
Rollback:
green
back to:
blue

☸️ **K8S REAL CHECKS**

Before switch:
kubectl get pods
Check:
Running
Ready
0 restarts
Then:
kubectl rollout status deployment/app-green
Logs:
kubectl logs deployment/app-green
Service:
kubectl get svc
Endpoints:
kubectl get endpoints
or modern Kubernetes:
kubectl get endpointslices

💔 **READINESS VS LIVENESS**

Very important interview concept.
Readiness
Question:

"Ee pod ki traffic ivvacha?"
If no:
Pod running
BUT
No traffic
Liveness
Question:
"Ee application process actually healthy ga unda?"
If liveness repeatedly fails:
Kubernetes restarts container
Example:

```
readinessProbe:
  httpGet:
    path: /health
    port: 8080

livenessProbe:
  httpGet:
    path: /health
    port: 8080
```

☁️ **PLATFORM-MANAGED BLUE-GREEN**
AWS Elastic Beanstalk example:
Blue Environment
↓
Production
Create:
Green Environment
↓
New version
Test Green.
Then swap environment URLs.
Concept:
Blue → Live
Green → Staging
After swap:
Green → Live
Blue → Previous version
Rollback:
Swap back

🔵 **AZURE APP SERVICE SLOTS**
Azure lo names usually:
Production slot
Staging slot
Think:
Production = Blue
Staging = Green
Deploy v2:
Staging
Test.
Then:
Swap
Now:
Staging version → Production
Rollback:
Swap back

📊 **MONITORING – ACTUAL JOB**
Deployment ayyaka monitoring is **not optional**.
You monitor four major things:
1. Availability
2. Errors
3. Performance
4. Capacity

1️⃣ **AVAILABILITY**
Question:
App users ki available unda?
Check:
ALB healthy targets

Pod readiness
HTTP health endpoint

2️⃣ **ERRORS**
Check:
HTTP 4xx
HTTP 5xx
Application exceptions
DB errors
Important:
5XX ↑
means server/application side issue possibility.

3️⃣ **PERFORMANCE**
Check:
Latency
Response time
CPU
Memory
DB latency
Example:
Before deployment:
Latency = 150ms
After Green:
Latency = 2.5 sec
Even if app technically works, deployment has a performance regression.

4️⃣ **CAPACITY**
Check:
CPU
Memory
Disk
Network
Suppose:
Blue CPU = 40%
Green CPU = 95%
Green may not be production-ready even if homepage works.

🚨 **CLOUDWATCH ALERT EXAMPLE**
Create alarm:
ALB 5XX > threshold
or:
Target response time > threshold
Then notification:
CloudWatch
↓
SNS
↓
Email / PagerDuty / Slack integration

🔐 **SECURITY MONITORING**
Now deployment successful.
Next question:
"Security okay na?"
Security monitoring means:
Who accessed?
What did they access?
Was access authorized?
Any suspicious activity?
Any exposed ports?
Any unusual traffic?

🔐 **EC2 LOGIN MONITORING**
Amazon Linux log locations/version can vary, but commonly SSH authentication events are in:
/var/log/secure
Search failed authentication:
sudo grep -i "failed" /var/log/secure
Watch:
sudo tail -f /var/log/secure
Login history:

last
For failed login history on systems using the relevant audit/login facilities:
lastb
if configured/available.

🚨 **SUSPICIOUS LOGIN**
Suppose:
Failed SSH attempts:
203.x.x.x
203.x.x.x
203.x.x.x
...
Hundreds of attempts.
Don't simply think:
"Block it in Security Group."
Because Security Groups don't provide explicit deny rules.
Better options:
AWS WAF
AWS Network Firewall
NACL where appropriate
Host firewall
Bastion/VPN/SSM architecture
And most importantly:
Don't expose SSH publicly unless necessary.

🔐 **SECURITY GROUP REAL-WORLD DESIGN**
Bad:
SSH
0.0.0.0/0
Anyone on internet can attempt SSH.
Better:
SSH
Company VPN CIDR
or use:
AWS Systems Manager Session Manager
For web:
ALB
↓
EC2
EC2 security group:
HTTP/HTTPS source = ALB SG
This is better than opening EC2 HTTP to everyone.

🔐 **IAM MONITORING**
IAM question:
"AWS lo evariki em permissions unnayi?"
Example:
Developer:
EC2 describe
CloudWatch read
Should not automatically have:
Delete production infrastructure
Use:
Least privilege
Meaning:
Required permission matrame ivvali.
Also:
Root account → MFA
Human users → MFA/SSO
Access keys → rotate/remove when unnecessary

🛡️ **ALB + WAF**
ALB public internet-facing.
Users request:
Internet
↓
ALB
Attack traffic kuda same route lo ravachu.
Add:
AWS WAF
Architecture:

Internet
↓
AWS WAF
↓
ALB
↓
EC2
WAF can help protect against common web attack patterns and unwanted traffic.
Examples:
SQL injection patterns
XSS patterns
Bad IPs
Rate-based abuse
Bot traffic
For DDoS, don't think WAF alone solves everything; AWS Shield and broader architecture can matter depending on threat level.

☸️ **KUBERNETES SECURITY**
Check:
kubectl get roles -A
kubectl get rolebindings -A
Also cluster-wide:
kubectl get clusterroles
kubectl get clusterrolebindings
Question:
"Evariki cluster lo em permission undi?"

🔐 **POD SECURITY**
Check:
kubectl get pods -o wide
Production lo ideally:
Don't unnecessarily run application as root.
SecurityContext use chestam.
Example:
securityContext:
runAsNonRoot: true
Also production workloads often consider:
readOnlyRootFilesystem
drop Linux capabilities
seccomp
resource limits
network policies
secret management

📦 **LOGGING ARCHITECTURE**
Real production lo logs only EC2 server lo store cheyyadam risky.
Typical:
Application
↓
Logs
↓
CloudWatch / ELK / Loki
↓
Dashboard
↓
Alert
Kubernetes:
Pod
↓
Container logs
↓
Log collector
↓
Loki / Elasticsearch / CloudWatch
↓
Grafana / Kibana

🚀 **HOTFIX SCENARIO**
Suppose Green v2 has tiny UI bug.
You don't need to redesign everything.
Workflow:
Developer fixes code

↓
Git commit
↓
CI pipeline
↓
Build
↓
Test
↓
Deploy Green v2.1
↓
Health check
↓
Functional test
↓
Traffic switch
If current Green already live:
Green v2
and hotfix:
Green v2.1
Don't directly modify production server manually if your organization follows immutable deployment practices.

🧠 **BLUE-GREEN vs ROLLING vs CANARY**
Interview lo very important.
Blue-Green
Blue = 100%
Green = 0%

Switch:

Blue = 0%
Green = 100%
Fast rollback.
Cost:
Two environments

Rolling Deployment
Old pods gradually replace avuthayi.
v1 v1 v1
↓
v2 v1 v1
↓
v2 v2 v1
↓
v2 v2 v2
Infrastructure duplication less.
Rollback more gradual/complex than a simple Blue-Green switch.

Canary
Small percentage first:
95% → Blue
5% → Green
Monitor.
If good:
75% → Blue
25% → Green
Then:
50 / 50
Then:
0 / 100
Canary is useful when you want to reduce blast radius.

🎯 **4-YEAR EXPERIENCE ANSWER**
Interviewer:
"How do you perform a zero-downtime Blue-Green deployment?"
You can answer:
"First I verify the current production Blue environment and establish a healthy baseline. I provision the Green environment with the same required capacity and configuration, deploy the new application version, and validate application health, dependencies, database compatibility, and security configuration. I register Green instances with a separate target group and wait until all health checks pass. Before switching traffic, I verify monitoring and rollback readiness. Then I switch the ALB listener from the Blue target group to Green. After the cutover, I closely monitor ALB 4xx/5xx, latency, target health, application logs, CPU, memory and business-

critical transactions. If we see a critical regression, I immediately switch traffic back to Blue and then investigate the Green environment. Once Green is stable for the agreed observation period, Blue can be decommissioned according to the rollback and retention policy."

Idi 4-year experience answer.

Not just:

"Blue old, Green new, switch traffic."

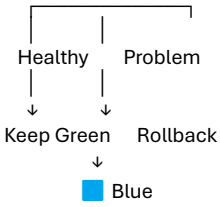
🧠 **MOST IMPORTANT REAL-WORLD POINTS**

Blue-Green deployment ante:

OLD ENVIRONMENT
+
NEW ENVIRONMENT
+
HEALTH CHECK
+
TEST
+
TRAFFIC SWITCH
+
MONITOR
+
FAST ROLLBACK

Remember this flow:

🟦 BLUE
Current Production
↓
🟩 GREEN CREATE
↓
Deploy New Version
↓
Health Check
↓
Functional Testing
↓
DB/API/Dependency Check
↓
Monitoring Ready
↓
🔥 TRAFFIC SWITCH
↓
🟩 GREEN LIVE
↓
Monitor
↓



One-line memory trick:

"Blue serves, Green prepares, Health Check verifies, Load Balancer switches, Monitoring watches, Blue stays ready for rollback."